

Lecture 9: The Fork and Exec sys calls.

Professor G. Sandoval

09



Some slides adapted by G. Sandoval for CS3224, from Tanenbaum & Bo, Modern Operating Systems: 4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved. Also some Slides by Brendan Dolan-Gavitt and Junfeng Yang @ Columbia.

Midterm

- Review: October 21
- October 23

Fork()

- **System Call** that creates an **exact copy** of current process called the child process.
- The child is initially a copy of the parent, but can be used to run a different branch of the program.
- After forking child and parent processes run in parallel.
- Any variables local to the parent process would have been **copied to the child process** so updating the variable on one process would not affect the other.



Fork() uses

- **Google Chrome** uses fork to handle each page within a separate process. This prevents client side code from bringing down the whole browser.
- Your **shell** uses fork() to run the programs you invoke from the command line.
- **Apache web server** uses fork() to create multiple server processes to handle requests.
- In general fork() is used to **spawn processes** in some parallel programs. Note this is different from threads since processes don't share the address space.

Fork example

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6
7  int main(void) {
8      int num = 10;
9      int pid = fork();
10
11     if( pid > 0){
12         printf("I'm the parent: My child has a pid: %d\n", pid);
13         pid = wait(NULL);
14         printf("My child is done\n");
15     }
16     else if (pid == 0) {
17         printf("I am the child: exiting\n");
18         exit(0);
19     }
20     return 0;
21 }
22 }
```

How is fork implemented?

- Program calls `fork()` system call
- The kernel:
 - duplicates the currently running process
 - sets the return value for the system call for original and duplicate (PID of the duplicate and 0, respectively)
 - puts both processes in the scheduler queue
 - 'returns' to each of the two programs.

Process Control Block in xv6

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
// Per-process state
struct proc {
    uint sz;                // Size of process memory (bytes)
    pde_t* pgdir;           // Page table
    char *kstack;           // Bottom of kernel stack for this process
    enum procstate state;   // Process state
    int pid;                // Process ID
    struct proc *parent;    // Parent process
    struct trapframe *tf;   // Trap frame for current syscall
    struct context *context; // switch() here to run process
    void *chan;             // If non-zero, sleeping on chan
    int killed;             // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;       // Current directory
    char name[16];          // Process name (debugging)
};
```

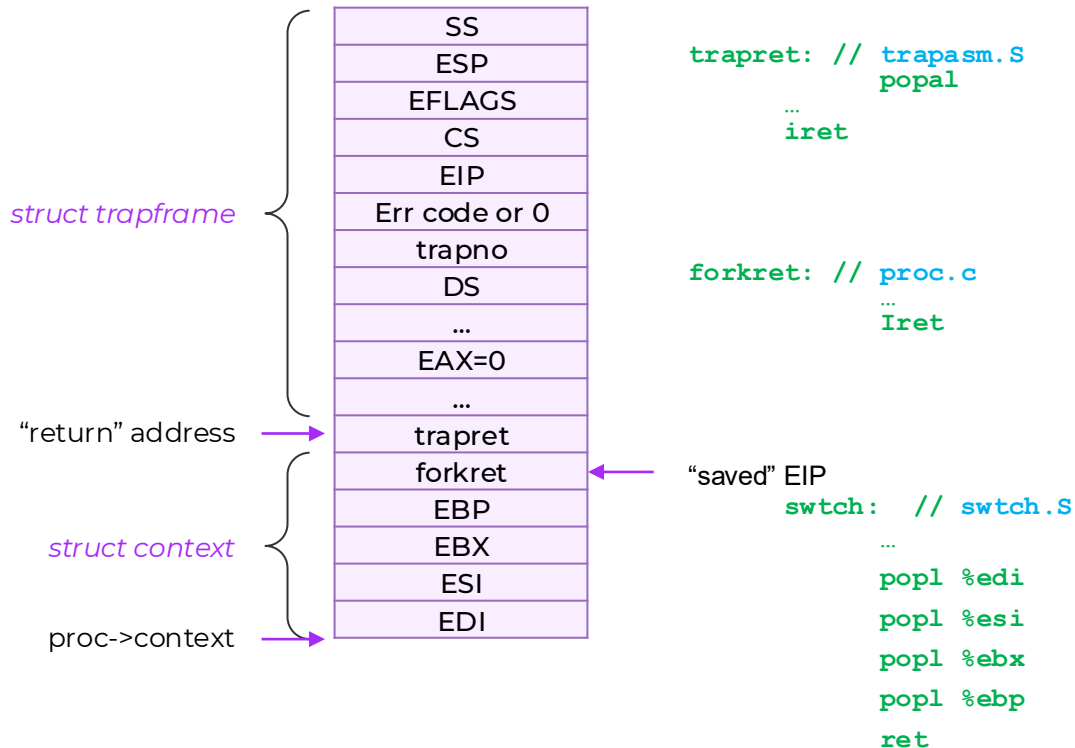
fork() Implementation

- `sysproc.c, proc.c`
- Allocate new PCB and stack
 - Set up EIP of child to `forkret` -> `trapret`
- Copy address space
 - Copy both page tables and physical pages
- Set parent pointer
- Copy parent's trap frame
- Change EAX in trap frame so that child returns 0
- Copy open file table

Summary of fork()

- **User space:** fork() → exec()
- **Kernel space:**
 - allocate PCB →
 - copy page tables →
 - schedule both →
 - replace memory image

Child process's kernel stack



XV6 fork Implementation

```
1 // Create a new process copying p as the parent.
2 // Sets up stack to return as if from system call.
3 // Caller must set state of returned proc to RUNNABLE.
4 int
5 fork(void)
6 {
7     [...]
8     // Allocate process.
9     if((np = allocproc()) == 0)
10         return -1;
11
12     // Copy process state from p.
13     np->pgdir = copyvm(proc->pgdir, proc->sz) == 0)
14     [...]
15
16     np->sz = proc->sz;
17     np->parent = proc;
18     *np->tf = *proc->tf;
19
20     // Clear %eax so that fork returns 0 in the child.
21     np->tf->eax = 0;
22
23     [...]
24
25     return pid;
26 }
```

Class 9: Which of these could actually happen?

c

```
printf("Parent\n");  
printf("Child\n");
```

Parent \n Child \n

0%

Child \n Parent \n

0%

Either

0%

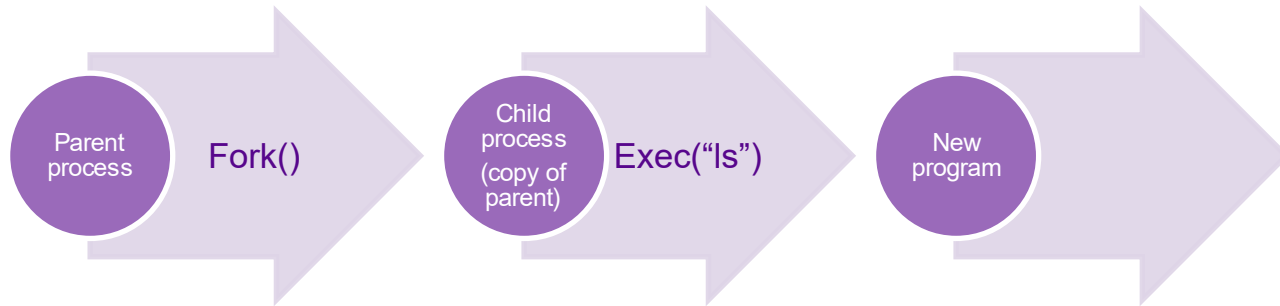
None of the above

0%

The Role of Exec

- If all we had was `fork()`, we couldn't launch new programs (why?)
- So we also have `exec()`, which allows you to replace the code and data in the current process with new code and data
- In modern UNIX systems, `exec` has many variants that provide different features:
 - Passing a variable number of arguments
 - Searching the directories listed in the `PATH` environment variable for the program

Fork() & exec()



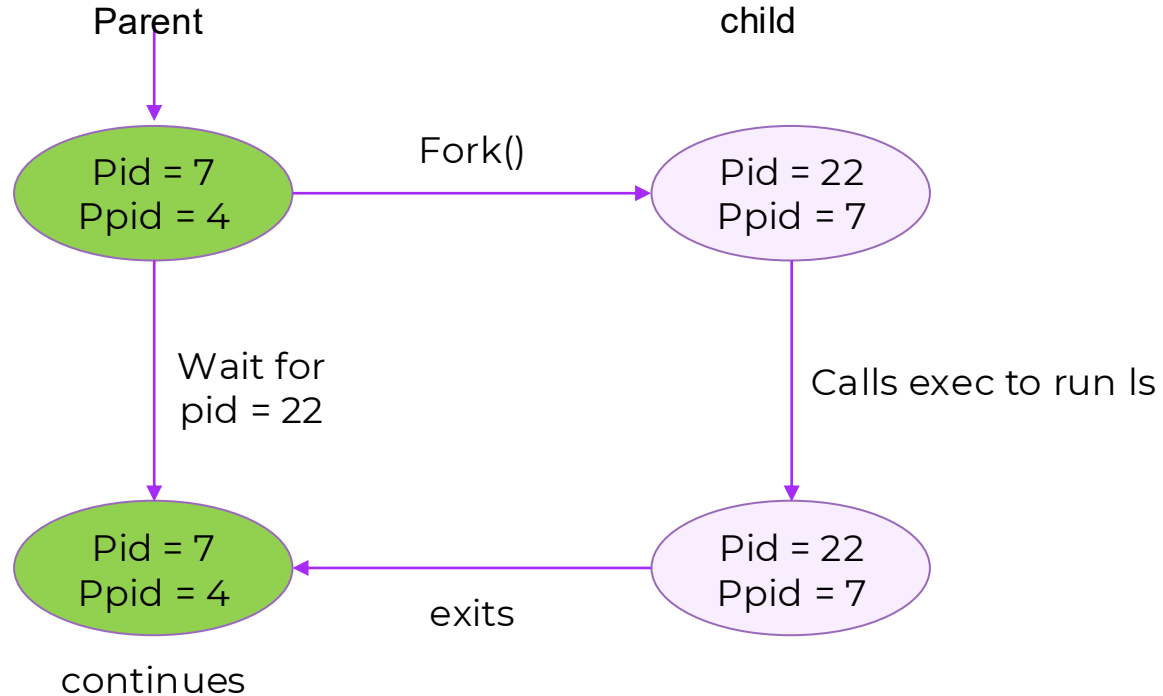
Bash example

- How would you use fork/exec in a shell?
- For example when the shell calls `ls ()`

Mini Shell code & demo

```
7   int main(void) {
8
9       char cmd[100];
10      while (1) {
11          printf("mini> ");
12          if (!fgets(cmd, sizeof(cmd), stdin)) break;
13          cmd[strcspn(cmd, "\n")] = '\0';
14          if (fork() == 0) {
15              execlp(cmd, cmd, NULL);
16              perror("exec");
17              exit(1);
18          }
19          wait(NULL);
20      }
21      return 0;
22  }
```

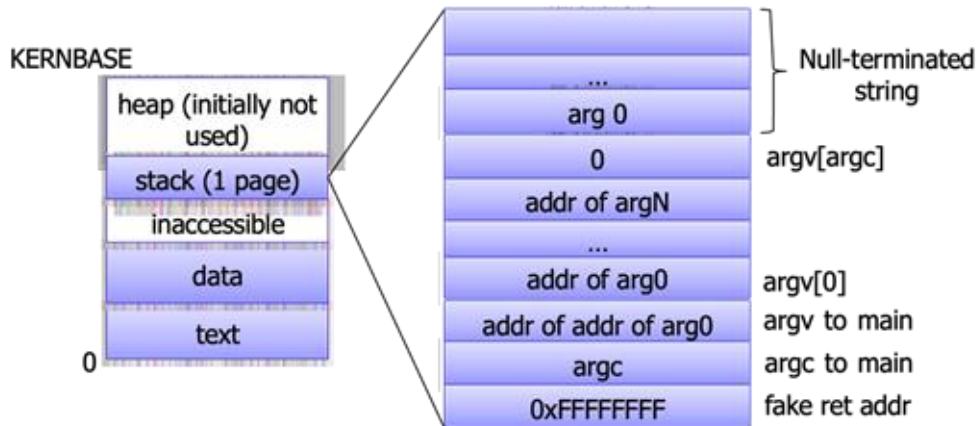
Bash shell example



exec()

Implementation

- sysfile.c, exec.c
- Set up user page table
- Load segments of the executable file into memory
- Set up stack and arguments to `main(int argc, char* argv[])`
- Jump to entry point (main) of the executable



In-Class Exercise

```
#include <stdio.h>
#include <unistd.h>
int main(void) {
    int i = 0;
    for (i = 0; i < 4; i++) {
        fork();
        printf("foo\n");
    }
    return 0;
}
```

How many times is "foo" printed?

Common Pitfalls

- Forgetting to call `wait()` → zombie processes
- Forgetting to `exit()` after failed `exec`
- Modifying shared resources (e.g., file descriptors) before `fork()`
- Expecting variables to stay in sync (they don't share memory)

Class 9: How many times is foo printed?

